

Semantic Smoothing for Language Models via Distribution Estimation and Embeddings

A Project Report

submitted by

SWATHI SHREE NARASHIMAN

*in partial fulfilment of requirements
for the award of the degree of*

BACHELOR OF TECHNOLOGY



**DEPARTMENT OF ELECTRICAL ENGINEERING
INDIAN INSTITUTE OF TECHNOLOGY MADRAS**

MAY 2026

THESIS CERTIFICATE

This is to certify that the thesis titled **SEMANTIC SMOOTHING FOR LANGUAGE MODELS VIA DISTRIBUTION ESTIMATION AND EMBEDDINGS**, submitted by **Swathi Shree N**, to the Indian Institute of Technology, Madras, for the award of the degree of **Bachelor of Technology**, is a bona fide record of the research work done by her under our supervision. The contents of this thesis, in full or in parts, have not been submitted to any other Institute or University for the award of any degree or diploma.

Prof. Andrew Thangaraj
Research Guide
Professor
Dept. of Electrical Engineering
IIT-Madras, 600 036

Place: Chennai

Date: 19th May 2026

ACKNOWLEDGEMENTS

I would like to express my sincere gratitude to Prof. Andrew Thangaraj for his invaluable guidance, encouragement, and support throughout my bachelor thesis.

His insightful suggestions, stimulating discussions, and perspective on research have greatly influenced my approach to problem-solving and independent thinking. Working under his supervision has been an immensely enriching experience.

I am especially grateful to Haricharan Balasundaram (EP21B015), a Dual Degree student, for mentoring me throughout the course of this thesis. His patience, guidance, and readiness to discuss ideas in depth have been instrumental to my learning experience and have significantly shaped my understanding of the project. I would also like to thank Pranay (EE22B132), with whom I have had the opportunity to work closely on this project.

I am also grateful to the members of the BTP evaluation committee for their constructive feedback and thought-provoking questions, which have greatly enriched my work.

I extend my heartfelt thanks to my parents for their unwavering encouragement, guidance and belief in my abilities. I also extend my heartfelt gratitude to my sister Charu and my brother-in-law Harish for setting the bar for academic excellence and ambition remarkably high. Finally, I am deeply appreciative of my friends especially Aravind Ramana, Nitin, and Ann Mary for their engaging discussions through graduate-level coursework and camaraderie. Meaningful interactions like these have played a vital role in broadening my perspective and deepening my understanding.

ABSTRACT

KEYWORDS: Distribution Estimation; Semantic Smoothing; Embeddings; Language Models

Semantic smoothing is a framework for improving language-model probability estimates by allowing statistically similar contexts to share information through semantic embeddings. Traditional smoothing techniques in language modeling primarily rely on frequency based adjustments to handle data sparsity, but they often ignore semantic relationships between contexts. In this work, we introduce a principled embedding-based smoothing method that leverages semantic similarity to transfer statistical strength across related contexts.

Our approach begins with a decomposition of the log-perplexity objective, which reveals that language model smoothing can be viewed as a collection of probability distribution estimation problems under Kullback-Leibler (KL) divergence loss. Building on this perspective, we analyze an embedding-based language generation model in which logits vary smoothly with respect to context embeddings. Under a Lipschitz-logit assumption, we show that if two contexts are close in embedding space, then their corresponding next word distributions are also close in KL divergence. This establishes a theoretical connection between semantic proximity and statistical similarity in language modeling.

Motivated by this observation, we formulate semantic smoothing as the problem of estimating a target distribution using both observed samples and auxiliary side information distributions that are known to be close in KL divergence. For a distribution over a d -symbol alphabet observed through n samples, and with access to side information lying within KL distance Δ of the target distribution, we propose a simple interpolation-based estimator. We prove that this estimator achieves worst-case KL risk of order

$$O(\min(\Delta, \frac{d}{n}))$$

demonstrating that semantic side information can substantially improve estimation accuracy when the auxiliary distributions are sufficiently close to the target. We further establish a matching-order minimax lower bound for the case of uniform side information, showing that the proposed estimator is order-optimal.

We also extend the framework to settings involving multiple semantically related distributions as well as practically relevant scenarios in which synonymous distributions must themselves be estimated from data. To evaluate the effectiveness of semantic smoothing, we conduct experiments on both synthetic Markov datasets and real world language modeling tasks using WikiText-103 bigram models. We incorporate semantic information derived from Word2Vec, GloVe, and GPT-2 embeddings, and apply our method on top of classical smoothing techniques such as Add-Constant smoothing and Kneser-Ney smoothing. Across all settings, semantic smoothing consistently achieves lower test perplexity, demonstrating that embedding based semantic sharing can significantly enhance statistical language modeling.

TABLE OF CONTENTS

ACKNOWLEDGEMENTS	i
ABSTRACT	1
INTRODUCTION	5
LANGUAGE MODELS	7
Maximum Likelihood Estimator	7
Add-Constant smoothing	7
Jelinek-Mercer Smoothing	8
Kneser-Ney smoothing	8
PERPLEXITY AND KL DECOMPOSITION	10
Perplexity	10
KL Divergence	10
Perplexity Decomposition	11
EMBEDDINGS AND DISTRIBUTIONAL PROXIMITY	12
Word Embeddings	12
Pre-trained Embeddings	12
Word2Vec	13
GloVe	14
Transformer Embeddings	14
DISTRIBUTION ESTIMATION WITH KL PROXIMITY	16
Embeddings to Next-Word Probabilities	16
KL Proximity	17
Minimax Theory	18
Extention to multiple synonymous distributions	19
Estimated synonymous distributions	20

SEMANTIC SMOOTHING IN PRACTICE	21
EXPERIMENTAL ANALYSIS	24
Experimental Setup	24
Analysis of Rare words	24
Initial Experiments with Jelinek-Mercer Smoothing	25
Semantic Interpolation for the Add-Constant Estimator	26
Cosine Similarity Based Semantic Smoothing	27
Towards Theoretically Motivated Semantic Smoothing	28
Results on Theoretical smoothing	29
Term-wise Interpolated Kneser-Ney Smoothing	31
CONCLUSIONS	34
LIST OF TABLES	35
LIST OF FIGURES	36

INTRODUCTION

A k -gram language model specifies a conditional distribution $\Pr(w_k \mid w_1, w_2, \dots, w_{k-1})$ for predicting the next word w_k given the preceding context $w_1, w_2, \dots, w_{k-1}$. Such models are typically trained on a large corpus of text and evaluated using *perplexity*, a metric closely related to log-loss that will be formally defined later. A fundamental challenge in language modeling is that test corpora inevitably contain k -grams that are absent from the training data. Consequently, naïve Maximum Likelihood (ML) estimates assign zero probability to unseen events, leading to prohibitively large test perplexities. This motivates the need for *smoothing* methods, which allocate nonzero probability mass to unseen k -grams Chen and Goodman (1996).

Classical smoothing techniques include i) add-constant smoothing due to Laplace Chen and Goodman (1996) and Lidstone Lidstone (1920), and interpolation or backoff-based approaches such as ii) Katz smoothing Katz (1987a), iii) Jelinek-Mercer smoothing Jelinek and Mercer (1980), and iv) Kneser-Ney smoothing Kneser and Ney (1995). These methods combine frequency statistics from lower-order l -grams, for $l = 1, \dots, k$, using heuristics motivated by natural language structure. Among these, Kneser-Ney smoothing is known to achieve particularly strong empirical performance across a range of benchmark datasets Chen and Goodman (1996). However, as the context length k increases, the computational and storage complexity of such models grows rapidly.

In recent years, large language models (LLMs) Vaswani *et al.* (2017) have significantly outperformed traditional n -gram approaches. Rather than relying directly on count statistics, LLMs map words and contexts into continuous embedding spaces and use deep neural architectures to transform context embeddings into next-word probability distributions. While highly effective, these models typically require extremely large training corpora and substantial computational resources.

In this work, we propose a new smoothing framework, termed *semantic smoothing*, that leverages semantic similarity between contexts through embeddings. Traditional smoothing methods share probability mass across lower-order contexts or frequency classes. In contrast, semantic smoothing transfers information between contexts whose embeddings are close, under the intuition that semantically similar contexts should induce similar next-word distributions. We justify this framework through the following three observations.

1) **Perplexity decomposition:** We show that the log-perplexity objective for a fixed context length can be decomposed into two terms: a model-independent empirical conditional entropy of the test sequence, and a weighted sum of Kullback–Leibler (KL) divergences between the true and estimated conditional distributions. As a result, minimizing empirical perplexity reduces to minimizing KL loss independently for each context-specific distribution estimation problem.

2) **Word embeddings and semantic proximity:** We quantify semantic similarity between contexts through distances in embedding space. Under a Lipschitz-logit assumption, satisfied by widely used embedding models such as Word2Vec Mikolov *et al.* (2013a,b) and GPT-2 Radford *et al.* (2018a), we prove that nearby context embeddings

correspond to nearby conditional next-word distributions in KL divergence.

3) **Semantic smoothing:** Combining the above insights, we develop a semantic smoothing framework for language models. Semantically related contexts for example, the words “big” and “enormous” in a bigram model are expected to induce similar next-word distributions, and can therefore share statistical information to improve estimation accuracy and reduce perplexity.

The resulting problem can be viewed more generally as distribution estimation under KL loss with side information: given an unknown distribution and an auxiliary distribution known to be close in KL divergence, how can one optimally combine the two sources of information?

Our main theoretical contribution addresses this question. Consider estimating a distribution over a size- d alphabet from n i.i.d. samples under KL loss, with access to a side-information distribution lying within KL distance Δ of the target distribution. We propose an interpolation estimator that linearly combines the add-constant estimator with the side-information distribution using carefully chosen weights. We show that this estimator achieves a minimax upper bound of

$$O\left(\min\left\{\Delta, \frac{d}{n}\right\}\right).$$

Furthermore, when the side-information distribution is uniform over the alphabet, we establish a matching-order minimax lower bound using Assouad’s lemma, demonstrating the order optimality of the proposed estimator.

To evaluate the practical effectiveness of semantic smoothing, we study the bigram setting. For each context word w , we identify m semantically related words $w_1, \dots, w_m$ as nearest neighbors in embedding space, and interpolate between the empirical conditional distribution $\Pr_{\text{train}}(\cdot \mid w)$ and the corresponding distributions $\Pr_{\text{train}}(\cdot \mid w_i)$ using weights motivated by the theory. We evaluate the proposed approach in three settings: (1) synthetic Markov data generated from low-rank transition structures that naturally induce state embeddings, (2) WikiText-103 using Word2Vec and GloVe embeddings in a bigram model, and (3) WikiText-103 using GPT-2 token embeddings in a token-level bigram model.¹

¹The code for the experiments is available at https://github.com/BaristaBandits/Distribution_Estimation.

LANGUAGE MODELS

Language models are staple in many domains including speech recognition, optical character and handwriting recognition. An Empirical Study of Smoothing Techniques for language models Chen and Goodman (1996) elaborates several widely used smoothing techniques for n-gram models.

A k -gram language model specifies a conditional distribution $\Pr(w_k \mid w_1, w_2, \dots, w_{k-1})$ for predicting the next word w_k given the preceding context $w_1, w_2, \dots, w_{k-1}$.

$$\Pr(w_1, w_2, \dots, w_T) = \prod_{t=1}^T \Pr(w_t \mid w_{t-k+1}, w_{t-k+2}, \dots, w_{t-1})$$

In this work, we focus on bi-gram models ($k=2$).

$$\Pr(w_1, w_2, \dots, w_T) = \prod_{t=1}^T \Pr(w_t \mid w_{t-1})$$

Maximum Likelihood Estimator

The naive estimator for bigram models is the Maximum Likelihood Estimator given by

$$\hat{P}(w_t \mid w_{t-1}) = \frac{\text{Count}(w_{t-1}, w_t)}{\sum_{w'} \text{Count}(w_{t-1}, w')} = \frac{\text{Count}(w_{t-1}, w_t)}{\text{Count}(w_{t-1})}.$$

While the Maximum Likelihood Estimator (MLE) is simple and intuitive, it has a key limitation: it assigns zero probability to any bigram that does not appear in the training corpus. This is problematic because it leads to overconfident models that cannot generalize to unseen word pairs. Formally, if $\text{Count}(w_{t-1}, w_t) = 0$, then

$$\hat{P}(w_t \mid w_{t-1}) = 0,$$

which makes the probability of any sentence containing such a bigram equal to zero.

Add-Constant smoothing

To address this issue, we introduce additive smoothing (also called Laplace smoothing or add- k smoothing), where a constant $k > 0$ is added to all bigram counts:

$$\hat{P}_k(w_t \mid w_{t-1}) = \frac{\text{Count}(w_{t-1}, w_t) + k}{\text{Count}(w_{t-1}) + k|V|},$$

where $|V|$ is the size of the vocabulary.

Although add- k (Laplace) smoothing avoids zero probabilities by redistributing probability mass uniformly, it has important limitations. In particular, it over-smooths frequent events and assigns too much probability mass to rare or unseen events, especially when the vocabulary size $|V|$ is large. As a result, it tends to underestimate the probability of frequent bigrams and perform poorly in practice for language modeling tasks.

Jelinek-Mercer Smoothing

To address these issues, a more effective approach is Jelinek–Mercer smoothing, which interpolates higher-order (bigram) estimates with lower-order (unigram) estimates. The idea is to combine the contextual model with a backoff distribution:

$$\hat{P}_\lambda(w_t \mid w_{t-1}) = \lambda \hat{P}_{\text{MLE}}(w_t \mid w_{t-1}) + (1 - \lambda) \hat{P}_{\text{MLE}}(w_t),$$

where $0 \leq \lambda \leq 1$ controls the interpolation between the bigram model and the unigram model.

Unlike add- k smoothing, Jelinek–Mercer smoothing does not uniformly redistribute probability mass; instead, it adapts by backing off to a lower-order model when contextual evidence is weak, leading to more robust estimates in sparse data settings.

While Jelinek–Mercer smoothing improves over add- k smoothing by interpolating higher and lower order models, it still has important limitations. In particular, it uses a fixed interpolation weight λ that is typically global (or only weakly context-dependent), which means it cannot fully adapt to the reliability of different contexts. Moreover, the unigram distribution $\hat{P}(w_t)$ used for backoff is purely based on raw frequency and does not capture how words behave in different contexts, leading to suboptimal estimates for rare events. Hence, the models described above do not adequately incorporate contextual information, an issue that is addressed by the Kneser-Ney estimator.

Kneser-Ney smoothing

A more effective smoothing method is Kneser-Ney smoothing, which improves upon this idea by using a more informative backoff distribution and better discounting of observed counts. Instead of relying on unigram frequencies, Kneser-Ney defines the backoff probability based on the number of unique contexts a word appears in:

$$\hat{P}_{\text{KN}}(w_t \mid w_{t-1}) = \frac{\max(\text{Count}(w_{t-1}, w_t) - D, 0)}{\text{Count}(w_{t-1})} + \lambda(w_{t-1}) P_{\text{cont}}(w_t),$$

where D is a discount parameter, $\lambda(w_{t-1})$ is a normalizing term ensuring probabilities sum to one, and $P_{\text{cont}}(w_t)$ is the continuation probability defined as

$$P_{\text{cont}}(w_t) = \frac{|\{w_{t-1} : \text{Count}(w_{t-1}, w_t) > 0\}|}{|\{(w_{t-1}, w') : \text{Count}(w_{t-1}, w') > 0\}|}.$$

This formulation captures not just how often a word appears, but how many different contexts it appears in, making Kneser-Ney smoothing particularly effective for language modeling.

Beyond Kneser-Ney smoothing, several other statistical estimators have been proposed to improve probability estimation in sparse data regimes. For instance, Good-Turing Good (1953) estimation reassigns probability mass based on the frequency of rare events by leveraging the number of n -grams that occur once, twice, and so on. Katz (1987b) backoff combines discounted maximum likelihood estimates with a back-off mechanism that recursively relies on lower order n -gram models when higher-order counts are unreliable. These methods primarily focus on redistributing probability mass more effectively under data sparsity, but still rely solely on frequency based statistics.

While statistical smoothing methods such as Good-Turing, Katz backoff, and Kneser-Ney significantly improve n -gram language models, they remain fundamentally limited by their reliance on local context windows and count-based statistics. They cannot effectively capture long range dependencies or rich semantic relationships between words beyond fixed length histories. Moreover, all the statistical models completely ignore the notion of synonymity, polysemy and semantic similarity between words.

A major shift in language modeling was introduced by Vaswani *et al.* (2017) with the Transformer architecture, which replaces recurrence and convolution with self-attention mechanisms. In modern formulations, tokens are first mapped into dense vector representations (embeddings), which serve as the input to the model. Transformers operate on these embeddings and enable models to attend over all positions in a sequence simultaneously. Thus allowing them to capture global dependencies and contextual interactions much more effectively than n -gram based approaches. This architecture forms the basis of modern large language models.

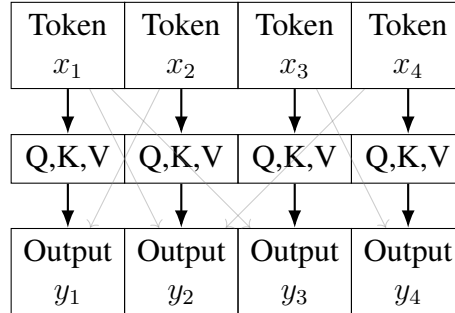


Figure 1: Simplified illustration of self-attention in Transformers, where each token attends to all other tokens in the sequence to produce contextualized representations.

In contrast to classical statistical language models, transformer-based models learn directly from these embedding representations in an end-to-end manner, rather than relying on explicit count-based probability estimates. As a result, they implicitly learn smoothing and generalization through parameterization and large scale training.

Motivated by the success of Transformers and the effectiveness of embedding-based semantic representations, we leverage the power of word embeddings in this work. Specifically, we focus on improving both add- k smoothing and the Kneser-Ney estimator by incorporating semantic information derived from word embeddings, allowing the model to capture richer similarities between words.

PERPLEXITY AND KL DECOMPOSITION

Perplexity

Perplexity is the standard metric used to evaluate probabilistic language models. It measures how well a model assigns probability to unseen test data. Intuitively, a lower perplexity indicates that the model is less "surprised" by the test sequence, and therefore provides better predictive performance.

A language model M is trained on a corpus x^S and evaluated on a disjoint test sequence x^T . A good model should generalize well from x^S to x^T , which requires assigning non-zero probability to previously unseen events. This motivates the need for smoothing techniques.

For a test sequence $x^T = (x_1, \dots, x_T)$, the perplexity of a model $\hat{p}$ is defined as

$$\text{PPL}(\hat{p}, x^T) = \exp \left(-\frac{1}{T} \sum_{t=1}^T \log \hat{p}(x_t \mid x_{<t}) \right).$$

Equivalently, perplexity is the exponential of the empirical cross-entropy between the true data distribution p and the model distribution $\hat{p}$:

$$\text{PPL}(\hat{p}) = \exp(H(p, \hat{p})).$$

KL Divergence

The Kullback-Leibler (KL) divergence is a measure of how one probability distribution differs from another. For two discrete probability distributions p and q defined over the same support, it is defined as

$$D_{\text{KL}}(p \parallel q) = \sum_x p(x) \log \frac{p(x)}{q(x)}.$$

Intuitively, $D_{\text{KL}}(p \parallel q)$ measures the inefficiency of using distribution q to approximate the true distribution p . It is always non-negative, and equals zero if and only if $p = q$ almost everywhere.

In language modeling, distributions are conditioned on contexts. This allows the KL divergence to be decomposed context-wise, showing that model performance depends on matching conditional distributions for each context.

Perplexity Decomposition

Since the true distribution p is unknown in practice, we estimate expectations using samples, leading to the empirical cross-entropy:

$$H(p, \hat{p}) = - \sum_x p(x) \log \hat{p}(x).$$

Corollary 1 (Perplexity Decomposition).

$$H(p, \hat{p}) = H(p) + D_{\text{KL}}(p \parallel \hat{p}).$$

where $H(p)$ is the entropy of the true distribution and D_{KL} is the KL divergence.

Therefore, minimizing perplexity is equivalent to minimizing cross-entropy, which in turn is equivalent to minimizing the KL divergence between the true and model distributions (up to an additive constant $H(p)$ independent of the model).

This observation reduces language modeling to a distribution estimation problem under KL loss. However, in practice, sparse data makes direct estimation unreliable, necessitating smoothing techniques such as add- k and Kneser-Ney smoothing. In the following sections, we introduce embeddings to incorporate semantic similarity into this estimation framework.

EMBEDDINGS AND DISTRIBUTIONAL PROXIMITY

Word Embeddings

Classical statistical language models treat words as discrete symbols and estimate probabilities purely from frequency counts. Such representations fail to capture semantic relationships between words. For example, the words “king” and “queen” or “dog” and “cat” are treated as completely independent tokens despite being semantically related.

Modern language models address this limitation using *word embeddings*, where each word is represented as a dense vector in a continuous vector space:

$$w \mapsto e_w \in \mathbb{R}^d.$$

Here, e_w denotes the embedding vector corresponding to the word w , and d is the embedding dimension.

The embedding space is learned such that semantically similar words are mapped to nearby vectors. Consequently, relationships between words can often be captured geometrically through distances and directions in the embedding space. For instance, words such as “king” and “queen” appear close together, while “dog” and “cat” cluster near the broader semantic concept of “animal”.

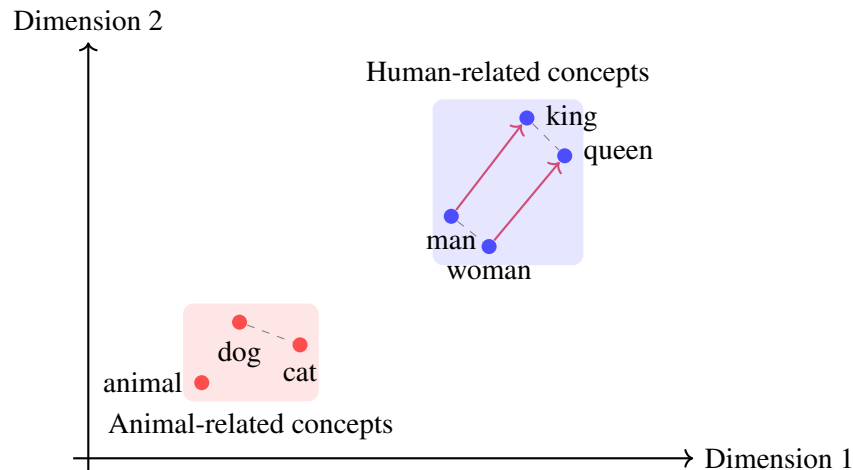


Figure 2: *Illustration of a word embedding space where semantically related words occupy nearby regions. Geometric relationships in the embedding space often capture semantic analogies and similarities.*

Pre-trained Embeddings

Among the most influential approaches for learning word embeddings are Word2Vec Mikolov *et al.* (2013b) and GloVe Pennington *et al.* (2014). Both methods learn dense

vector representations of words from large text corpora, but they differ fundamentally in how semantic relationships are extracted from language data.

Word2Vec

Word2Vec is a predictive embedding model that learns word representations by exploiting local contextual information. The central idea behind Word2Vec is the distributional hypothesis: words appearing in similar contexts tend to have similar meanings. The model therefore learns embeddings by training a neural network to predict contextual relationships between words occurring in natural text.

Two standard architectures are commonly used in Word2Vec: the Continuous Bag of Words (CBOW) model and the Skip Gram model. The CBOW model predicts a target word from its surrounding context words, whereas the Skip Gram model predicts surrounding context words given a center word. In practice, the Skip Gram architecture is particularly effective for capturing semantic relationships and learning high-quality representations for rare words.

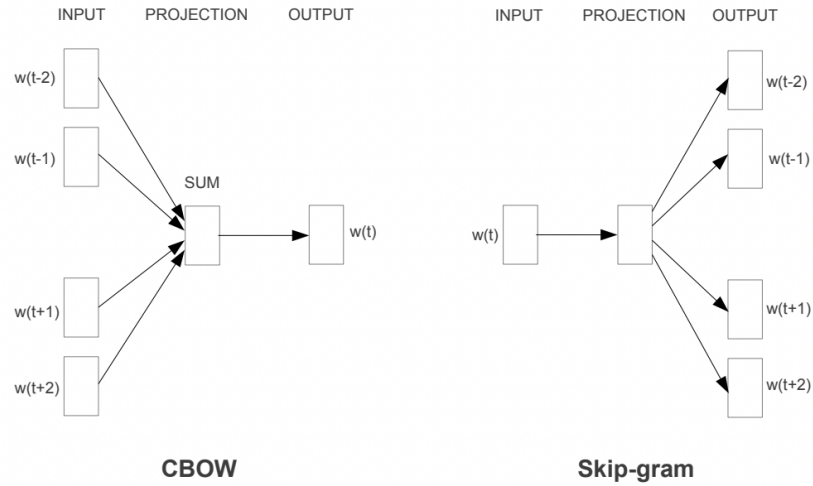


Figure 3: Word2Vec architectures. *Source: Mikolov et al. (2013b)*

a sequence of words $(w_1, w_2, \dots, w_T)$ and a context window of size m , the Skip-Gram objective seeks to maximize

$$\frac{1}{T} \sum_{t=1}^T \sum_{\substack{-m \leq j \leq m \\ j \neq 0}} \log P(w_{t+j} | w_t).$$

The conditional probability of observing a context word w_O given a center word w_I is modeled using the softmax function:

$$P(w_O | w_I) = \frac{\exp(e_{w_O}^\top e_{w_I})}{\sum_{w \in V} \exp(e_w^\top e_{w_I})},$$

where e_{w_I} and e_{w_O} denote the embedding vectors associated with the input and output words respectively, and V represents the vocabulary.

Training encourages words that occur in similar contexts to develop similar vector representations. As a result, semantic structure emerges naturally in the embedding space. Remarkably, linear relationships between embeddings often encode meaningful semantic analogies such as

$$e_{\text{king}} - e_{\text{man}} \approx e_{\text{queen}} - e_{\text{woman}}.$$

In practice, computing the full softmax denominator over large vocabularies is computationally expensive. To address this, Word2Vec commonly employs approximations such as negative sampling or hierarchical softmax, which significantly reduce training complexity while preserving embedding quality.

GloVe

GloVe (Global Vectors for Word Representation) adopts a different philosophy for learning embeddings. Instead of directly predicting neighboring words, GloVe leverages global statistical information derived from word co-occurrence counts across the entire corpus. The method is motivated by the observation that ratios of co-occurrence probabilities encode meaningful semantic relationships between words.

Let X_{ij} denote the number of times word j appears in the context of word i . GloVe constructs embeddings by factorizing the co-occurrence matrix through the optimization problem

$$J = \sum_{i,j=1}^{|V|} f(X_{ij}) \left(e_i^\top \tilde{e}_j + b_i + \tilde{b}_j - \log X_{ij} \right)^2,$$

where e_i and $\tilde{e}_j$ represent word and context embeddings respectively, b_i and $\tilde{b}_j$ are bias terms, and $f(X_{ij})$ is a weighting function that reduces the influence of extremely frequent word pairs.

The objective seeks to ensure that the inner product between embedding vectors approximates the logarithm of the corresponding co-occurrence count. Consequently, words that frequently occur in similar contexts obtain nearby vector representations in the embedding space.

Unlike Word2Vec, which primarily relies on local contextual prediction, GloVe explicitly incorporates global corpus statistics during training. Nevertheless, both methods produce semantically meaningful embedding spaces that capture syntactic regularities, semantic similarity, and latent relationships between words.

Transformer Embeddings: GPT-2

The Generative Pretrained Transformer (GPT-2) Radford *et al.* (2018b) is a transformer-based autoregressive language model that learns contextual embeddings using the self-attention mechanism introduced in transformers Vaswani *et al.* (2017). Unlike Word2Vec and GloVe, which assign a single fixed vector representation to each word, GPT-2 generates embeddings that depend on the surrounding context in which a token appears.

GPT-2 first converts raw text into smaller units called *tokens* using Byte Pair Encoding

(BPE). BPE is a subword tokenization algorithm that iteratively merges frequently occurring character sequences to form a vocabulary of commonly used subwords. This allows GPT-2 to efficiently handle rare words, unseen words, and morphological variations while maintaining a manageable vocabulary size.

After tokenization using Byte Pair Encoding (BPE), each token is mapped to a learned embedding vector through an embedding matrix

$$E \in \mathbb{R}^{|V| \times d},$$

where $|V|$ denotes the vocabulary size and d is the embedding dimension. For a token w_i , its embedding is obtained by indexing the corresponding row of the embedding matrix:

$$w_i \mapsto e_{w_i} \in \mathbb{R}^d.$$

In this work, we use these learned token embeddings directly rather than the contextual hidden representations produced by later transformer layers. Consequently, each token or subword is associated with a fixed vector representation learned during GPT-2 pretraining.

DISTRIBUTION ESTIMATION WITH KL PROXIMITY

Embeddings to Next-Word Probabilities: Lipschitzness

Transformer-based language models convert context embeddings into probability distributions over the vocabulary through a sequence of linear and nonlinear transformations. Given a context embedding

$$f_c \in \mathbb{R}^t,$$

the model computes a score, or *logit*, for each word $w \in V$ using an inner product of the form

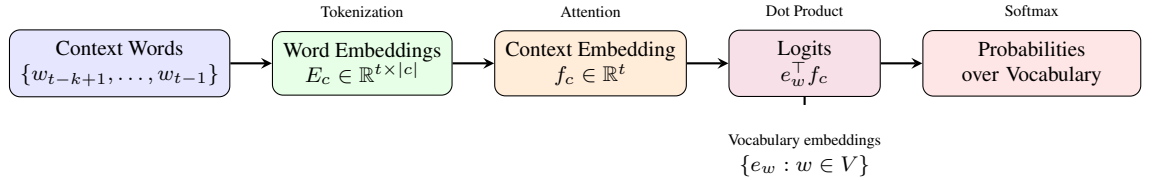
$$e_w^\top f_c,$$

where e_w denotes the embedding associated with the word w .

The logits are subsequently transformed into a probability distribution using the softmax function:

$$P(w|c) = \frac{\exp(e_w^\top f_c)}{\sum_{y \in V} \exp(e_y^\top f_c)}.$$

The softmax operation converts arbitrary real-valued scores into valid probabilities that sum to one. Words with larger logits are assigned higher probabilities.



Motivated by the remarkable success of transformer-based language models, we introduce a generalized probabilistic model that captures the relationship between embeddings and next-word probabilities.

Definition 2 ((L, ϵ) -Lipschitz-logit process). *A random process on a vocabulary V with conditional probability distribution $P(\cdot|c)$ for every context c is said to be an (L, ϵ) -Lipschitz-logit process if*

$$P(w|c) = \frac{\exp(g_w(f_c) + \epsilon_{c,w})}{\sum_{y \in V} \exp(g_y(f_c) + \epsilon_{c,y})}, \quad (1)$$

where:

- $f_c \in \mathbb{R}^t$ is the embedding associated with context c ,
- $g_w : \mathbb{R}^t \rightarrow \mathbb{R}$ is an L -Lipschitz function satisfying

$$|g_w(f_c) - g_w(f_{\tilde{c}})| \leq L \|f_c - f_{\tilde{c}}\|_2,$$

- and the perturbation term satisfies

$$|\epsilon_{c,w}| \leq \epsilon.$$

The Lipschitz condition formalizes the intuition that small perturbations in embedding space should not produce large changes in the output logits. Consequently, nearby contexts in embedding space are expected to induce similar next-word probability distributions.

Transformer-based language models naturally satisfy this condition. In particular, when

$$g_w(f_c) = e_w^\top f_c,$$

the Lipschitz constant is given by

$$L = \max_{w \in V} \|e_w\|_2.$$

KL Proximity

The Lipschitz-logit framework establishes a direct connection between geometric proximity in embedding space and probabilistic proximity between next-word distributions.

Intuitively, if two contexts possess nearby embeddings, then their logits should also be similar. Since the softmax transformation is continuous and smooth, the resulting conditional probability distributions should likewise remain close. To quantify this notion of closeness, we use the Kullback–Leibler (KL) divergence introduced earlier. The following theorem formalizes this relationship.

Theorem 3 (KL proximity from embedding proximity). *Let c and $\tilde{c}$ be two contexts in an (L, ϵ) -Lipschitz-logit process with embeddings f_c and $f_{\tilde{c}}$ respectively. Then,*

$$D_{\text{KL}}(P(\cdot|c) \| P(\cdot|\tilde{c})) \leq 2L\|f_c - f_{\tilde{c}}\|_2 + 4\epsilon.$$

The theorem shows that proximity in embedding space directly controls proximity under KL divergence. Thus, semantically similar contexts induce similar next-word probability distributions.

In many practical embedding architectures, the context embedding function itself satisfies a Lipschitz condition. Specifically, suppose

$$\|f_c - f_{\tilde{c}}\|_2 \leq L_f \|E_c - E_{\tilde{c}}\|_F,$$

where $\|\cdot\|_F$ denotes the Frobenius norm.

Such conditions hold for several commonly used embedding methods, including Word2Vec Continuous Bag-of-Words (CBOW) embeddings and transformer-based embeddings with bounded input norms. Under this assumption, the KL proximity theorem can be expressed directly in terms of word embeddings.

Corollary 4. *If the context embedding function is L_f -Lipschitz, then*

$$D_{\text{KL}}(P(\cdot|c) \| P(\cdot|\tilde{c})) \leq 2LL_f \|E_c - E_{\tilde{c}}\|_F + 4\epsilon.$$

This result demonstrates that semantic similarity between contexts, as measured through word embeddings, induces proximity between the corresponding next-word distributions.

In practice, embedding models often use larger contexts than the language model itself. For example, embeddings may be learned using two-sided contexts, whereas a bigram language model conditions only on the immediately preceding word.

Suppose the embedding context is $c = (w_{i+j} : j \in I_k)$, where $I_k = \{\pm 1, \dots, \pm(k-1)\}$, while the language model conditions only on a subset $c_S = (w_{i+j} : j \in S)$, $S \subseteq I_k$.

The KL proximity framework naturally extends to such settings.

Corollary 5. *For contexts restricted to a subset S ,*

$$D_{\text{KL}}(P(\cdot|c_S) \parallel P(\cdot|\tilde{c}_S)) \leq 2LL_f \|E_{c_S} - E_{\tilde{c}_S}\|_F + 4\epsilon.$$

For the special case of a bigram language model, $S = \{-1\}$, and therefore $E_{c_S} = e_{w_{i-1}}$. Consequently,

$$D_{\text{KL}}(P(\cdot|w_{i-1}) \parallel P(\cdot|\tilde{w}_{i-1})) \leq 2LL_f \|e_{w_{i-1}} - e_{\tilde{w}_{i-1}}\|_2 + 4\epsilon.$$

The results developed in this section establish the theoretical foundation for semantic smoothing. Specifically, they show that semantically similar contexts possess nearby next-word probability distributions under KL divergence. This motivates the use of embedding similarity as side information in distribution estimation and smoothing methods for language models.

Minimax Theory

Consider n i.i.d. samples $X^n = (X_1, X_2, \dots, X_n)$ drawn from a discrete distribution $\pi = (\pi_1, \pi_2, \dots, \pi_d)$. Let $N_i = \sum_{j=1}^n I(X_j = i)$ denote the number of occurrences of symbol i , where $I(\cdot)$ is the indicator function. The empirical estimator of π is then

$$\hat{\pi}_i^{\text{emp}} = \frac{N_i}{n}.$$

A central problem in information theory and language modeling is estimating the unknown distribution π from finite samples under KL divergence loss. Classical minimax results show that the variable add-constant estimator π^* achieves asymptotically optimal performance under expected KL loss Braess and Sauer (2004).

Theorem 6. *Braess and Sauer (2004) For the minimax optimal estimator π^* ,*

$$\max_{\pi} \mathbb{E} [D_{\text{KL}}(\pi \parallel \pi^*(X^n))] = \frac{d-1}{2n} + o\left(\frac{1}{n}\right).$$

This result captures the fundamental estimation error that arises due to finite sampling. In particular, the error decreases roughly as d/n , where d is the alphabet size and n is the number of observed samples.

In practice, however, additional side information may be available. Suppose that along with samples from π , we also know that π is close to another distribution $\tilde{\pi} = (\tilde{\pi}_1, \tilde{\pi}_2, \dots, \tilde{\pi}_{d_0})$ in KL divergence. More precisely, assume

$$\pi \in B_{\text{KL}}(\tilde{\pi}, \Delta) = \{\pi' : D_{\text{KL}}(\pi' \| \tilde{\pi}) \leq \Delta\},$$

where Δ measures the closeness between the two distributions.

In the language modeling setting, $\pi = P(\cdot|c)$, $\tilde{\pi} = P(\cdot|\tilde{c})$, where c and $\tilde{c}$ are semantically similar contexts in the embedding space. The KL proximity results developed earlier imply that neighboring contexts often induce nearby conditional distributions. Thus, semantic similarity naturally provides side information for distribution estimation.

The corresponding minimax problem becomes

$$\min_{\hat{\pi}} \max_{\pi \in B_{\text{KL}}(\tilde{\pi}, \Delta)} \mathbb{E} [D_{\text{KL}}(\pi \| \hat{\pi}(X^n))] . \quad (2)$$

A simple and effective strategy is to interpolate between the empirical estimator and the synonymous distribution $\tilde{\pi}$.

Theorem 7. *The interpolation estimator*

$$\hat{\pi} = \alpha \pi^*(X^n) + (1 - \alpha) \tilde{\pi},$$

for an appropriate choice of $\alpha \in [0, 1]$, satisfies

$$\max_{\pi \in B_{\text{KL}}(\tilde{\pi}, \Delta)} \mathbb{E} [D_{\text{KL}}(\pi \| \hat{\pi})] \leq \min \left(\frac{d-1}{2n}, \Delta \right) .$$

This theorem gives an important insight. If the synonymous distribution $\tilde{\pi}$ is very close to π , then interpolation can significantly reduce estimation error. Thus, semantic similarity can directly improve probability estimation.

Extension to multiple synonymous distributions

In practice, a context may have several neighboring contexts in the embedding space. Suppose we have synonymous distributions $\pi^{(1)}, \pi^{(2)}, \dots, \pi^{(m)}$ with corresponding KL proximity bounds $\Delta_1, \Delta_2, \dots, \Delta_m$.

The estimator can then be generalized to

$$\hat{\pi} = \alpha_0 \pi^* + \sum_{i=1}^m \alpha_i \pi^{(i)}, \quad (3)$$

where $\alpha_i \geq 0$, $\sum_{i=0}^m \alpha_i = 1$.

The interpolation weights are chosen according to the reliability of each synonymous distribution. In practice, smoother interpolation is obtained using a softmax weighting

rule:

$$\alpha_i \propto \begin{cases} \exp\left(-\tau \frac{d-1}{2n}\right), & i = 0, \\ \exp(-\tau \Delta_i), & i \geq 1, \end{cases}$$

where τ is a temperature parameter.

This assigns larger weights to semantically closer contexts while still allowing contributions from multiple neighbors.

Estimated synonymous distributions

So far, we assumed that the synonymous distribution $\tilde{\pi}$ is known exactly. In practice, however, it must also be estimated from data. Suppose that $X^n \sim \pi$, $Y^{n_0} \sim \tilde{\pi}$, where Y^{n_0} consists of samples from a synonymous context. We first estimate $\tilde{\pi}$ using the minimax optimal estimator $\tilde{\pi}^*(Y^{n_0})$, and then interpolate:

$$\hat{\pi} = \alpha \pi^*(X^n) + (1 - \alpha) \tilde{\pi}^*(Y^{n_0}).$$

The following theorem provides the resulting estimation bound.

Theorem 8. *For an appropriate choice of α ,*

$$\mathbb{E}[D_{\text{KL}}(\pi \parallel \hat{\pi})] \lesssim \min\left(\frac{d-1}{2n}, \Delta + \log\left(1 + \frac{d_0}{n_0}\right) + \log 2\right).$$

Ignoring logarithmic lower-order terms, the bound behaves approximately as

$$\min\left(\frac{d}{n}, \Delta + \frac{d_0}{n_0}\right).$$

This result forms the theoretical foundation of semantic smoothing. If semantically related contexts have nearby conditional distributions, then probability estimation can be improved by interpolating information across neighboring contexts in the embedding space.

SEMANTIC SMOOTHING IN PRACTICE

The theoretical developments of the previous sections establish a direct connection between semantic similarity in embedding spaces and proximity of conditional probability distributions in KL divergence. In particular, Theorem 3 demonstrates that if two contexts are close in the embedding space, then the corresponding next-word probability distributions are also close in KL distance. This observation provides the fundamental motivation for semantic smoothing: information from semantically related contexts can be transferred in a principled manner to improve probability estimation.

Consider a training corpus represented by the sequence $u^S = (u_1, u_2, \dots, u_S)$ and a test sequence $v^T = (v_1, v_2, \dots, v_T)$, both assumed to be generated from a common random language process $\mathcal{P}$. We assume that $\mathcal{P}$ satisfies the (L, ϵ) -Lipschitz-logit condition introduced in the previous section. For every context c , let $p_{\mathcal{P}}(\cdot|c)$ denote the true next-word distribution associated with the process. Further, let e_w denote the embedding corresponding to a word w , and let f_c denote the embedding associated with the context c .

The embeddings are assumed to be available along with the training corpus and are used to construct a language model M with parameters $\theta(M)$. Given the training sequence u^S , the model produces estimated conditional probabilities $\hat{p}_{M,u^S}(\cdot|c)$, which are then used for predicting words in the test sequence. The overall objective of the language model is to minimize the test perplexity $(\hat{p}_{M,u^S}, v^T)$.

From a theoretical perspective, we consider random training and test sequences

$$U^S \sim \mathcal{P}, \quad V^T \sim \mathcal{P},$$

and study the optimization problem

$$\min_{\theta(M)} \min_{U^S, V^T} [\log ((\hat{p}_{M,U^S}, V^T))] . \quad (4)$$

Using the perplexity decomposition established earlier in Corollary 1, the expected log-perplexity can be expressed as a sum of two terms: an entropy term independent of the model and a KL-divergence term measuring the mismatch between the model distribution and the true distribution. Consequently, minimizing perplexity reduces to minimizing the KL divergence between the estimated conditional distributions and the true next-word distributions for every context.

Thus, for each context c , the language modeling problem can be viewed as the following distribution estimation problem:

$$\min_{\theta(M)} \min_{U^S} [(p_{\mathcal{P}}(\cdot|c) \parallel \hat{p}_{M,U^S}(\cdot|c))] . \quad (5)$$

At this stage, the embedding-based KL proximity results become crucial. Suppose that for a given context c , there exist neighboring contexts $c_1, c_2, \dots, c_m$ whose embeddings

are close to f_c in the embedding space. By Theorem 3, the corresponding probability distributions $p_{\mathcal{P}}(\cdot|c_i)$ are also close to $p_{\mathcal{P}}(\cdot|c)$ in KL divergence. Therefore, these neighboring contexts contain useful statistical information that can aid estimation of the target distribution.

This idea is especially important in sparse-data regimes. Classical smoothing methods such as add- k smoothing or Kneser-Ney smoothing rely primarily on frequency statistics and backoff heuristics. However, they do not explicitly exploit semantic similarity between contexts. As a result, two semantically similar contexts with different surface forms are treated independently even when their next-word distributions are expected to be similar.

Semantic smoothing addresses this limitation by allowing probability mass to be shared across semantically related contexts. Instead of relying solely on counts from the target context, the model incorporates information from neighboring contexts in the embedding space. Consequently, rare or unseen contexts can borrow statistical strength from semantically similar contexts that occur more frequently in the training corpus.

For a fixed training sequence $U^S = u^S$, let $\hat{p}_{M,u^S}(\cdot|c)$ denote the probability estimate produced by the underlying language model for context c . Using the theoretical framework developed earlier, and in particular Theorem 8, we define the semantically smoothed estimator as a convex interpolation of the estimates corresponding to neighboring contexts:

$$\begin{aligned} \hat{p}_{M,u^S}^{\text{sem}}(\cdot|c) = & \alpha_c^{(c)} \hat{p}_{M,u^S}(\cdot|c) \\ & + \alpha_{c_1}^{(c)} \hat{p}_{M,u^S}(\cdot|c_1) + \cdots + \alpha_{c_m}^{(c)} \hat{p}_{M,u^S}(\cdot|c_m). \end{aligned} \quad (6)$$

Here,

$$\alpha_c^{(c)}, \alpha_{c_1}^{(c)}, \dots, \alpha_{c_m}^{(c)} \in [0, 1]$$

are interpolation coefficients satisfying

$$\alpha_c^{(c)} + \sum_{i=1}^m \alpha_{c_i}^{(c)} = 1.$$

The coefficient $\alpha_c^{(c)}$ controls the contribution of the original context estimate, while the coefficients $\alpha_{c_i}^{(c)}$ determine the influence of neighboring contexts. Intuitively, contexts whose embeddings are closer to f_c should receive larger interpolation weights since their corresponding distributions are expected to be closer in KL divergence.

In practice, the neighboring contexts are obtained using nearest-neighbor search in the embedding space. Given a context embedding f_c , one computes its nearest contexts according to cosine similarity or Euclidean distance. The interpolation weights may then be chosen using similarity-based kernels such as

$$\alpha_{c_i}^{(c)} \propto \exp(-\lambda \|f_c - f_{c_i}\|_2), \quad (7)$$

where $\lambda > 0$ is a temperature parameter controlling the strength of semantic smoothing.

The proposed framework is highly general and can be applied on top of different underlying smoothing schemes. For example, the base estimator $\hat{p}_{M,u^S}(\cdot|c)$ may correspond

to an add- k estimator, a Kneser-Ney estimator, or any other statistical language model. Semantic smoothing then acts as an additional embedding-aware regularization layer that incorporates semantic proximity information into the estimation process.

Thus, the overall methodology combines the statistical robustness of classical smoothing methods with the semantic generalization capabilities of modern embedding representations. This provides a principled bridge between classical count-based language models and modern representation learning approaches used in transformer-based language models.

EXPERIMENTAL ANALYSIS

Experimental Setup

To evaluate the effectiveness of semantic smoothing, experiments were conducted on the WikiText-2 and WikiText-103 corpora. For WikiText-103, we used the standard train-test split available through the Python `datasets` library Lhoest *et al.* (2021), while ignoring the validation split. A subset consisting of 100,000 training sentences and 10,000 test sentences was used in order to enable rapid experimentation and hyperparameter exploration. After preprocessing and tokenization, the resulting corpus contained 66,753 unique words. Standard preprocessing steps included converting all text to lowercase, replacing numerical values and URLs with special tokens, removing punctuation and non-ASCII characters, and tokenizing sentences using whitespace splitting for Word2Vec and GloVe experiments.

All experiments were performed using bigram language models together with embedding-based semantic interpolation methods. Word embeddings were obtained using pre-trained embedding models such as GloVe (100 dimensions) Pennington *et al.* (2014) and Word2Vec (300 dimensions) Mikolov *et al.* (2013a), and nearest neighbors in the embedding space were computed using cosine similarity. Throughout the experiments, rare words were defined as words occurring exactly once in the training corpus. In further experiments, we also used GPT-2 tokenization and embeddings (768 dimensions) Radford *et al.* (2018a) to evaluate our semantic smoothing framework on transformer-based embedding models.

Analysis of Rare Words and Embedding Coverage

An initial exploratory analysis was conducted on the WikiText-2 corpus in order to understand the extent to which semantic neighbors are available for rare words. Rare words occur only once in the corpus. The following statistics were obtained:

Statistic	Value
Total rare words	27390
Rare words with embeddings	19922
Average number of neighbors	35.91
Median number of neighbors	50

Table 1: *Distribution of rare words and semantic neighbors in the dataset.*

Neighbor threshold	≥ 0	≥ 1	≥ 2	≥ 5	≥ 10	≥ 20
Number of rare words	19922	19775	19532	18588	17132	14825

Table 2: *Distribution of semantic neighbors among rare words in the corpus.*

These statistics provide an important insight into the feasibility of semantic smoothing. A large fraction of rare words possess meaningful neighboring words in the embedding space, and most rare words have several semantically related neighbors available. In particular, the median number of neighbors being **50** indicates that semantic neighborhoods are generally dense even for infrequent words.

This observation strongly supports the central hypothesis of this work: although count-based estimators suffer from data sparsity for rare words, embeddings provide additional semantic side information that can potentially compensate for the lack of frequency statistics. Thus, semantic smoothing can leverage neighboring contexts and related words to improve probability estimation in low-data regimes.

Initial Experiments with Jelinek-Mercer Smoothing

The first set of experiments incorporated semantic interpolation into the Jelinek–Mercer smoothing framework. Recall that Jelinek–Mercer smoothing interpolates between higher-order and lower-order language models using fixed interpolation coefficients.

In our preliminary experiments, the interpolation weights were modified using semantic similarity information derived from embeddings. Two approaches were explored:

1. interpolation using normalized count frequencies.
2. interpolation using normalized cosine similarity scores.

For a target word w with synonymous neighbors $\mathcal{N}(w) = \{w_1, w_2, \dots, w_m\}$ the normalized count-based interpolation coefficient was defined as

$$\lambda_i = \frac{\text{Count}(w_i)}{\sum_{j=1}^m \text{Count}(w_j)}. \quad (8)$$

Similarly, cosine similarity based interpolation weights were computed as

$$\lambda_i = \frac{\cos(e_w, e_{w_i})}{\sum_{j=1}^m \cos(e_w, e_{w_j})}, \quad (9)$$

where e_w and e_{w_i} denote the embeddings of the target word and its neighboring word respectively.

However, despite incorporating semantic information, the improvements obtained using Jelinek-Mercer smoothing were limited. One possible explanation is that Jelinek-Mercer smoothing already performs interpolation between unigrams and bigrams, and therefore the additional semantic interpolation on bigram probabilities may not significantly alter the effective probability distribution.

These experiments nevertheless provided useful insights into how semantic information interacts with classical smoothing techniques.

Semantic Interpolation for the Add-Constant Estimator

Following the limited gains obtained with Jelinek-Mercer smoothing, attention was shifted toward the add-constant estimator. Unlike Jelinek-Mercer smoothing, the add-constant estimator relies entirely on local count statistics and therefore suffers more severely from sparsity. Consequently, it provides a more suitable setting for incorporating semantic side information.

Initially, interpolation was applied only to rare words. The underlying intuition was that common words already possess sufficiently reliable count statistics, whereas rare words are the primary source of estimation error.

A threshold parameter, was introduced to determine whether semantic interpolation should be applied. If the count of a word exceeded the threshold, no semantic interpolation was performed. Otherwise, semantic smoothing was applied using neighboring words from the embedding space.

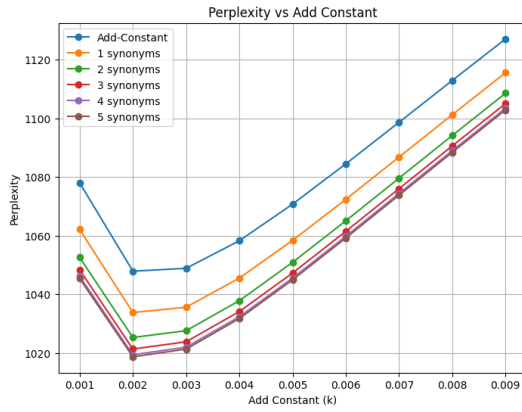


Figure 4: *Count-based Semantic interpolation on words occurring once under the add-constant estimator on Wikitext2*

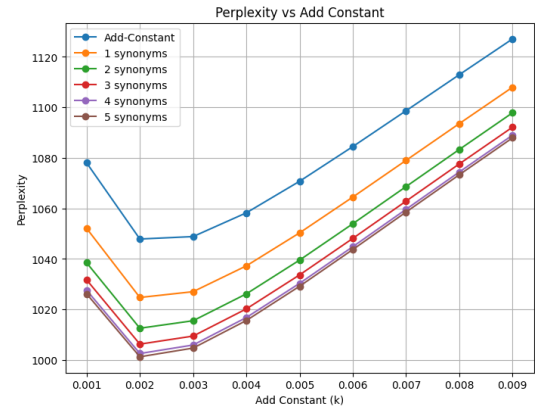


Figure 5: *Count-based Semantic interpolation on words occurring less than 100 times under the add-constant estimator on Wikitext2*

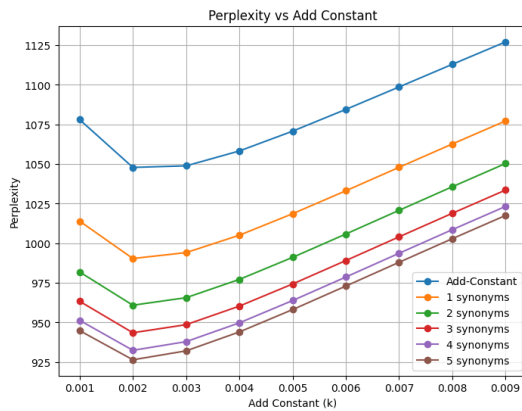


Figure 6: *Count-based Semantic interpolation on words occurring less than 100 times under the add-constant estimator on Wikitext2*

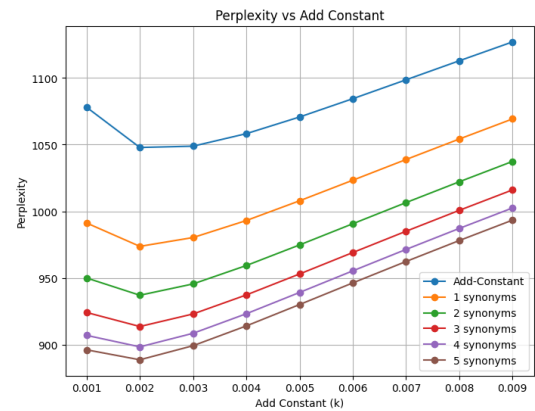


Figure 7: *Count-based Semantic interpolation on words occurring less than 1000 times under the add-constant estimator on Wikitext2*

Initially, very small threshold values were used so that only extremely rare words were semantically smoothed. Subsequently, the threshold was gradually increased to include increasingly larger portions of the vocabulary.

Surprisingly, the experiments revealed that increasing the threshold consistently improved performance, roughly improving the perplexity by **200** for higher thresholds. In other words, semantic interpolation remained beneficial even for words that were not particularly rare. This was a significant empirical observation.

The result suggests that semantic smoothing does not merely compensate for missing counts; rather, it acts as a more general regularization mechanism for language modeling. Semantically related words provide useful contextual information even when direct frequency estimates are available. Consequently, the experiments indicated that smoothing the entire vocabulary may be preferable to selectively smoothing only rare words.

This observation aligns closely with the theoretical KL proximity framework developed earlier. If semantically similar contexts possess nearby probability distributions in KL distance, then borrowing statistical information from neighboring contexts should improve estimation more broadly rather than only in extreme sparsity regimes.

Cosine Similarity Based Semantic Smoothing

After the success of count-normalized interpolation, cosine similarity based semantic smoothing was explored in greater detail. In this setting, the interpolation weights were directly proportional to embedding similarity scores. The smoothing was implemented on all words.

For a target word w and neighboring words $w_1, w_2, \dots, w_m$, the interpolation coefficients were defined as

$$\lambda_i = \frac{\cos(e_w, e_{w_i})}{\sum_{j=1}^m \cos(e_w, e_{w_j})}, \quad (10)$$

where τ is a temperature hyperparameter controlling the sharpness of the interpolation distribution.

Cosine similarity based interpolation assigns larger weights to highly similar neighboring words while still permitting information transfer from moderately related words.

Although this approach consistently improved perplexity compared to the baseline add-constant estimator (improvement roughly **30** perplexity units), the gains were smaller than those obtained using count-based interpolation. This observation suggested that cosine similarity alone may not fully capture the underlying semantic relationships relevant for probability estimation, motivating the search for more theoretically grounded interpolation strategies beyond simple embedding similarity.

We observed similar results on Word2Vec embeddings as well. We reproduced the results on Wikitext103 which showed similar trends in perplexity. However, since Wikitext103 is a denser corpus than Wikitext2 we observe that the baseline perplexity is lower.

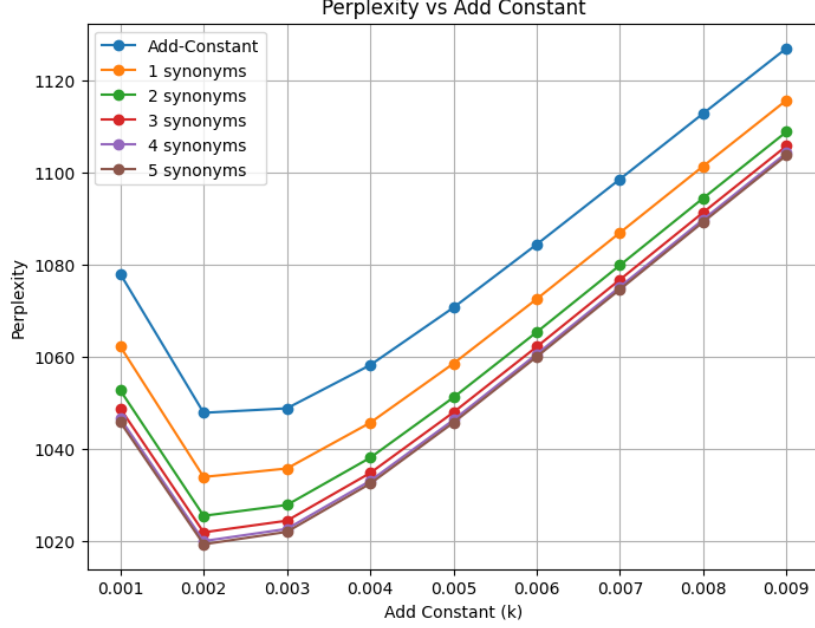


Figure 8: *Cosine similarity based semantic interpolation for all words under the add-constant estimator on Wikitext2*

Towards Theoretically Motivated Semantic Smoothing

The exploratory experiments described above demonstrated that embedding-based interpolation consistently improves classical smoothing estimators, particularly the add-constant estimator. However, the interpolation schemes used initially were primarily heuristic in nature and were not directly derived from the KL proximity theory developed earlier.

These observations motivated the development of a more theoretically informed semantic smoothing framework. In particular, the empirical success of interpolation suggested that semantically similar words indeed possess nearby conditional distributions, thereby validating the assumptions underlying the Lipschitz-logit model and KL proximity analysis.

Suppose the context c occurs n_c times in the training corpus u^S , and a neighboring context c_i occurs n_{c_i} times. Let d_c denote the number of distinct next words observed following c , and similarly let d_{c_i} denote the number of distinct next words observed following c_i . In practice, when these quantities are unknown, they are estimated directly from the observed next-word samples in the corpus.

From the KL proximity results developed earlier, semantically nearby contexts induce nearby next-word distributions. In particular, by the KL proximity theorem,

$$D_{\text{KL}}(P(\cdot|c) \| P(\cdot|c_i)) \leq 2L\|f_c - f_{c_i}\|_2 + 4\epsilon.$$

Motivated by this bound, we estimate the semantic proximity between contexts c and c_i as

$$\Delta_{cc_i} = 2L\|f_c - f_{c_i}\|_2 + 4\epsilon, \quad (11)$$

where L is the Lipschitz constant and $\|\cdot\|_2$ denotes the Euclidean norm in the embedding space.

The minimax interpolation results further show that the estimation error depends both on the sampling uncertainty and the KL proximity of neighboring distributions. In particular, Theorem 8 suggests balancing:

$$\frac{d_c}{n_c} \quad \text{and} \quad \Delta_{cc_i} + \frac{d_{c_i}}{n_{c_i}}.$$

Using the interpolation framework for multiple synonymous distributions, we define interpolation coefficients $\alpha_{c'}^{(c)}$ through loss proxies associated with each context:

$$\alpha_{c'}^{(c)} \propto \begin{cases} \phi\left(\frac{d_c-1}{2n_c}\right), & c' = c, \\ \phi\left(\Delta_{cc'} + \log\left(1 + \frac{\beta d_{c'}}{n_{c'}}\right) + \left[\log\left(\frac{1}{\beta}\right)\right]_+\right), & c' = c_i, \end{cases} \quad (12)$$

where β is the add- β smoothing parameter, $(x)_+ = \max(x, 0)$ and $\phi(x)$ is a decreasing weighting function (like the exponential function).

Thus, contexts with smaller estimated KL divergence and more reliable empirical statistics receive larger interpolation weights. The resulting semantically smoothed estimator combines information from multiple neighboring contexts in a theoretically motivated manner.

For any language model M , these loss proxies can be computed using the model’s estimated next-word distributions together with embedding-based KL proximity estimates. Consequently, semantic smoothing can be applied as a post processing layer on top of arbitrary language models to obtain improved next-word probability estimates.

Results on Theoretical smoothing

We next evaluated the theoretically motivated semantic smoothing framework derived from the KL proximity results and minimax interpolation theory. For a bigram language model, the context embedding f_c reduces to the word embedding e_w . Using the KL proximity theorem, we estimated the semantic proximity between two words w and $\tilde{w}$ as

$$\Delta_{w\tilde{w}} = 5\|e_w - e_{\tilde{w}}\|_1,$$

where the constant 5 was chosen based on the maximum embedding norm observed across the vocabulary. For every context word $w \in V$, the effective alphabet size d_w of next words following w was estimated using the Chebyshev polynomial estimator proposed in Wu and Yang (2015). The synonymous neighbors of a word were selected according to the minimax-inspired loss proxy

$$\Delta_{w\tilde{w}} + \log\left(1 + \frac{\beta \hat{d}_{\tilde{w}}}{n_{\tilde{w}}}\right),$$

motivated by the upper bounds from the KL proximity interpolation theory. We used $\beta = 0.005$, selected the top $m = 50$ nearest synonymous contexts, and used interpolation weights proportional to $\phi(x) = 1/x$. Words without available embeddings were excluded from semantic smoothing.

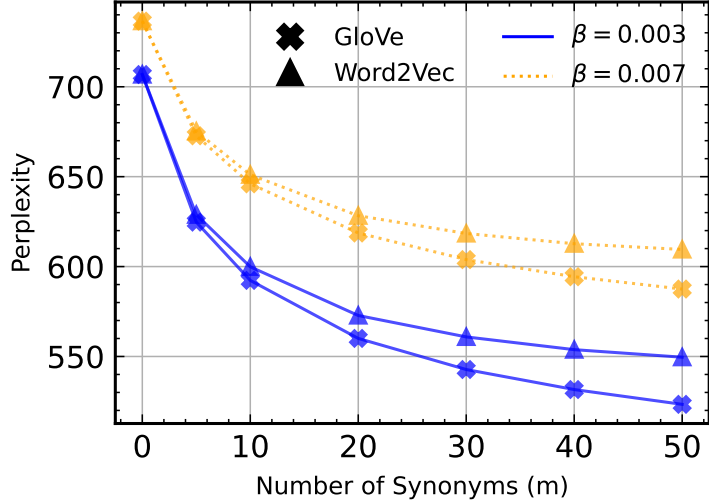


Figure 9: *Semantic Smoothing under the add-constant estimator with GloVe and Word2Vec embeddings on Wikitext103*

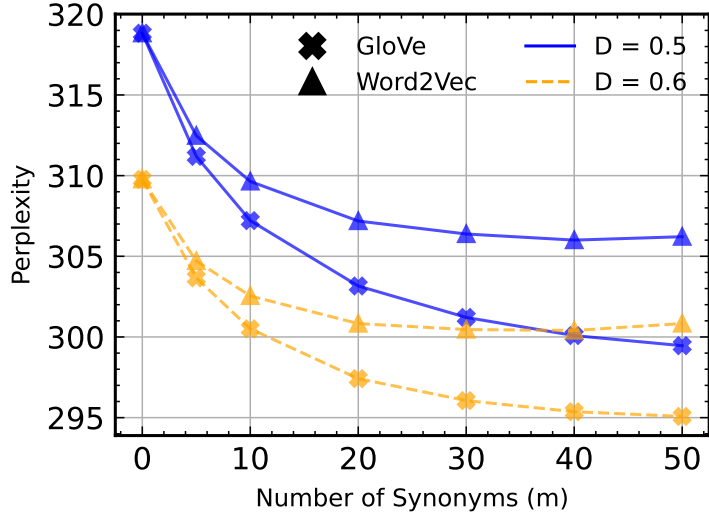


Figure 10: *Semantic Smoothing under the add-constant estimator with GloVe and Word2Vec embeddings on Wikitext103*

This theoretically informed interpolation strategy resulted in substantial perplexity improvements for both the add-constant and Kneser-Ney (KN) estimators. While the KN estimator achieved lower absolute perplexity values overall, semantic smoothing consistently improved performance across both models. Furthermore, the improvements became stronger as the number of synonymous neighbors increased, validating the theoretical prediction that semantically proximal contexts can effectively reduce estimation error under KL loss. Similar trends were observed using both GloVe and Word2Vec embeddings.

The gains persisted even when moving to GPT-2 embeddings and tokenization. The baseline perplexity in this setting was already significantly lower due to the use of Byte Pair Encoding (BPE) tokenization and transformer-based embeddings, which naturally reduce sparsity and improve probability estimation. Despite this stronger baseline, semantic smoothing continued to provide consistent improvements, demonstrating that the proposed framework generalizes effectively across both classical word-level em-

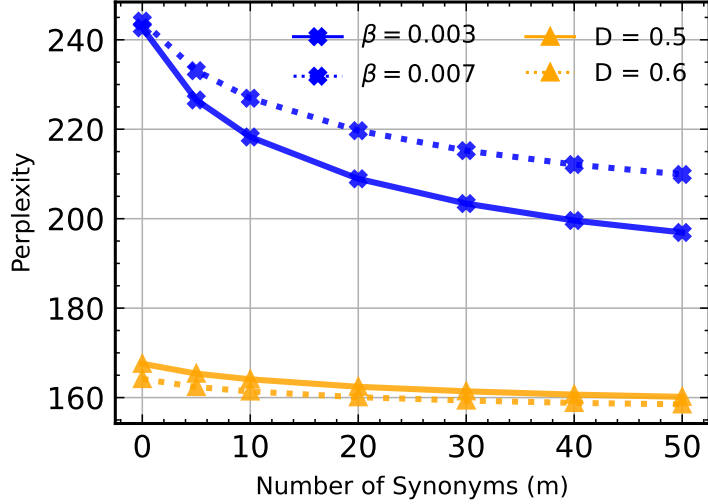


Figure 11: *Semantic Smoothing under the add-constant and Kener-Ney estimator with GPT2 embeddings on Wikitext103*

beddings and transformer-based tokenization schemes.

Estimator	Embedding	Parameter	$m = 0$	$m = 1$	$m = 2$	$m = 5$	$m = 10$	$m = 20$	$m = 50$
Add- β	GloVe	$\beta = 0.003$	706.79	624.72	592.31	560.04	542.75	531.61	523.38
		$\beta = 0.005$	719.72	649.21	620.38	591.15	575.38	565.22	557.81
		$\beta = 0.007$	736.46	672.73	646.03	618.68	603.87	594.35	587.49
	Word2Vec	$\beta = 0.003$	706.79	628.80	599.87	572.79	560.88	553.80	549.59
		$\beta = 0.005$	719.72	652.42	626.61	602.30	591.69	585.49	581.91
		$\beta = 0.007$	736.46	675.17	651.04	628.27	618.40	612.71	609.50
	GPT-2	$\beta = 0.003$	242.68	226.54	218.26	208.92	203.37	199.58	196.92
		$\beta = 0.005$	241.90	229.06	222.16	214.16	209.32	205.96	203.58
		$\beta = 0.007$	244.16	233.09	226.93	219.67	215.22	212.11	209.88
Naive KN	GloVe	$D = 0.5$	318.83	311.19	307.21	303.16	301.21	300.09	299.46
		$D = 0.6$	309.77	303.66	300.51	297.42	296.06	295.37	295.08
		$D = 0.7$	303.65	298.70	296.18	293.85	292.95	292.61	292.59
	Word2Vec	$D = 0.5$	318.83	312.47	309.64	307.19	306.38	306.00	306.21
		$D = 0.6$	309.77	304.71	302.54	300.83	300.46	300.40	300.83
		$D = 0.7$	303.65	299.55	297.89	296.74	296.71	296.91	297.51
	GPT-2	$D = 0.5$	167.56	165.35	164.08	162.42	161.40	160.66	160.17
		$D = 0.6$	164.15	162.37	161.37	160.08	159.34	158.81	158.49
		$D = 0.7$	162.10	160.63	159.83	158.82	158.29	157.92	157.73

Table 3: *Perplexity values for semantic smoothing with increasing number of synonyms ($m = \{0, 1, 2, 5, 10, 20, 50\}$) across different embedding models and smoothing estimators.*

Term-wise Interpolated Kneser-Ney Smoothing

We additionally explored a more structured semantic interpolation strategy for the Kneser-Ney (KN) estimator by interpolating its individual components separately rather than smoothing the final probability distribution directly. Recall that the KN estimator can be decomposed as

$$P_{\text{KN}}^D(w_i|w_{i-1}) = p_1^D(w_i|w_{i-1}) + \lambda(w_{i-1}) p_2(w_i),$$

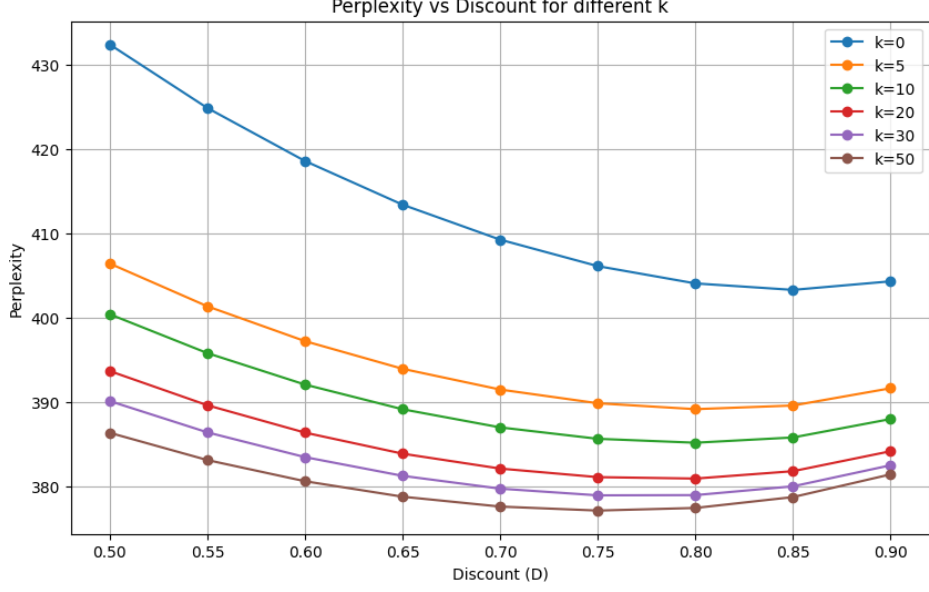


Figure 12: *Semantic term-wise smoothing under Kneser-Ney estimator using GloVe embeddings on Wikitext103*

where p_1^D denotes the discounted bigram probability, $\lambda(w_{i-1})$ is the backoff weight, and $p_2(w_i)$ is the continuation probability.

Motivated by the observation that different terms capture different semantic structures, we performed semantic interpolation independently for each component. The discounted conditional probability and backoff weights were interpolated using semantic neighbors of the context word w_{i-1} , while the continuation probability was interpolated using semantic neighbors of the target word w_i . This yielded semantically smoothed components of the form

$$p_{1\text{-sem}}^D(\cdot|w), \quad \lambda_{\text{sem}}(w), \quad p_{2\text{-sem}}(w),$$

obtained through weighted interpolation across synonymous words in the embedding space.

The resulting semantically smoothed KN score was then computed as

$$s_{\text{KN-sem}}^D(w_i|w_{i-1}) = p_{1\text{-sem}}^D(w_i|w_{i-1}) + \lambda_{\text{sem}}(w_{i-1}) p_{2\text{-sem}}(w_i).$$

Since independent interpolation of the constituent terms does not automatically guarantee a valid probability distribution, we applied an empirical normalization step to ensure that the scores summed to one over the vocabulary. Under this empirical normalization procedure, the term-wise interpolated KN model consistently improved perplexity relative to the baseline KN estimator. The improvements further increased with the number of semantic neighbors used, suggesting that separately smoothing the different structural components of KN captures semantic relatedness more effectively than direct interpolation of the final probabilities alone.

Several additional approaches were also explored in an attempt to further improve semantic smoothing performance. These included fine-tuning pretrained GloVe embeddings directly on the target corpus as well as learning embeddings from a custom

lightweight transformer model trained on the dataset. While these methods produced meaningful changes in the embedding geometry, they did not lead to consistent or significant improvements in perplexity over the pretrained embedding based approaches.

CONCLUSIONS

We proposed semantic smoothing, a framework for improving language model probability estimation by transferring information across semantically similar contexts in the embedding space. Unlike classical smoothing methods that rely primarily on frequency counts, semantic smoothing uses embedding geometry as side information to better estimate next-word distributions, especially in sparse data settings.

We established a theoretical connection between embedding proximity and KL proximity of next-word distributions through the Lipschitz-logit framework, and formulated a minimax distribution estimation problem with side information. The resulting interpolation based estimators provide an information theoretic justification for semantic smoothing.

Experimentally, semantic smoothing consistently improved perplexity for both add-constant and Kneser-Ney estimators across Word2Vec, GloVe, and GPT-2 embeddings. The gains persisted even in transformer tokenized settings with already strong baselines. The experiments also showed that smoothing the entire vocabulary was more effective than targeting only rare words, and that structured interpolation methods such as term-wise Kneser-Ney smoothing also yielded improvements.

Several directions remain open. Extending semantic smoothing to longer contexts and transformer architectures requires scalable nearest neighbor search and efficient interpolation in high dimensional embedding spaces. Learning interpolation weights or embeddings specifically optimized for semantic smoothing may further improve performance. Finally, the underlying distribution estimation problem with KL proximity side information appears to be of independent theoretical interest, particularly when the side-information distribution itself must be estimated from samples.

LIST OF TABLES

1	<i>Distribution of rare words and semantic neighbors in the dataset. . .</i>	24
2	<i>Distribution of semantic neighbors among rare words in the corpus.</i>	24
3	<i>Perplexity values for semantic smoothing with increasing number of synonyms ($m = \{0, 1, 2, 5, 10, 20, 50\}$) across different embedding models and smoothing estimators.</i>	31

LIST OF FIGURES

1	Simplified illustration of self-attention in Transformers, where each token attends to all other tokens in the sequence to produce contextualized representations.	9
2	<i>Illustration of a word embedding space where semantically related words occupy nearby regions. Geometric relationships in the embedding space often capture semantic analogies and similarities.</i>	12
3	Word2Vec architectures. <i>Source: Mikolov et al. (2013b)</i>	13
4	<i>Count-based Semantic interpolation on words occurring once under the add-constant estimator on Wikitext2</i>	26
5	<i>Count-based Semantic interpolation on words occurring less than 100 times under the add-constant estimator on Wikitext2</i>	26
6	<i>Count-based Semantic interpolation on words occurring less than 100 times under the add-constant estimator on Wikitext2</i>	26
7	<i>Count-based Semantic interpolation on words occurring less than 1000 times under the add-constant estimator on Wikitext2</i>	26
8	<i>Cosine similarity based semantic interpolation for all words under the add-constant estimator on Wikitext2</i>	28
9	<i>Semantic Smoothing under the add-constant estimator with GloVe and Word2Vec embeddings on Wikitext103</i>	30
10	<i>Semantic Smoothing under the add-constant estimator with GloVe and Word2Vec embeddings on Wikitext103</i>	30
11	<i>Semantic Smoothing under the add-constant and Kneser-Ney estimator with GPT2 embeddings on Wikitext103</i>	31
12	<i>Semantic term-wise smoothing under Kneser-Ney estimator using GloVe embeddings on Wikitext103</i>	32

REFERENCES

1. **Braess, D.** and **T. Sauer** (2004). Bernstein polynomials and learning theory. *Journal of Approximation Theory*, **128**(2), 187–206. ISSN 0021-9045. URL <https://www.sciencedirect.com/science/article/pii/S0021904504000723>.
2. **Chen, S. F.** and **J. Goodman**, An empirical study of smoothing techniques for language modeling. *In 34th Annual Meeting of the Association for Computational Linguistics*. Association for Computational Linguistics, Santa Cruz, California, USA, 1996. URL <https://aclanthology.org/P96-1041/>.
3. **Good, I. J.** (1953). The population frequencies of species and the estimation of population parameters. *Biometrika*, **40**(3/4), 237–264. ISSN 00063444, 14643510. URL <http://www.jstor.org/stable/2333344>.
4. **Jelinek, F.** and **R. L. Mercer**, Interpolated estimation of Markov source parameters from sparse data. *In Proceedings of the Workshop on Pattern Recognition in Practice*. North-Holland, Amsterdam, The Netherlands, 1980.
5. **Katz, S. M.** (1987a). Estimation of probabilities from sparse data for the language model component of a speech recognizer. *IEEE Transactions on Acoustics, Speech, and Signal Processing*, **35**(3), 400–401.
6. **Katz, S. M.** (1987b). Estimation of probabilities from sparse data for the language model component of a speech recognizer. *IEEE Transactions on Acoustics, Speech, and Signal Processing*, **35**(3), 400–401.
7. **Kneser, R.** and **H. Ney**, Improved backing-off for m-gram language modeling. *In Proceedings of the IEEE International Conference on Acoustics, Speech, and Signal Processing (ICASSP)*, volume 1. 1995.
8. **Lhoest, Q., A. Villanova del Moral, Y. Jernite, et al.**, Datasets: A community library for natural language processing. *In Proceedings of EMNLP: System Demonstrations*. 2021. URL <https://aclanthology.org/2021.emnlp-demo.21/>.

9. **Lidstone, G. J.** (1920). Note on the general case of the Bayes-Laplace formula for inductive or a posteriori probabilities. *Transactions of the Faculty of Actuaries*, **8**, 182–192.
10. **Mikolov, T., K. Chen, G. Corrado, and J. Dean** (2013a). Efficient estimation of word representations in vector space. URL <https://arxiv.org/abs/1301.3781>.
11. **Mikolov, T., I. Sutskever, K. Chen, G. S. Corrado, and J. Dean**, Distributed representations of words and phrases and their compositionality. In **C. Burges, L. Bottou, M. Welling, Z. Ghahramani, and K. Weinberger** (eds.), *Advances in Neural Information Processing Systems*, volume 26. Curran Associates, Inc., 2013b. URL https://proceedings.neurips.cc/paper_files/paper/2013/file/9aa42b31882ec039965f3c4923ce901b-Paper.pdf.
12. **Pennington, J., R. Socher, and C. Manning**, GloVe: Global vectors for word representation. In **A. Moschitti, B. Pang, and W. Daelemans** (eds.), *Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing (EMNLP)*. Association for Computational Linguistics, Doha, Qatar, 2014. URL <https://aclanthology.org/D14-1162/>.
13. **Radford, A., K. Narasimhan, T. Salimans, and I. Sutskever** (2018a). Improving language understanding by generative pre-training. *OpenAI*. URL https://cdn.openai.com/research-covers/language-unsupervised/language_understanding_paper.pdf.
14. **Radford, A., K. Narasimhan, T. Salimans, and I. Sutskever** (2018b). Improving language understanding by generative pre-training. *OpenAI*. URL <https://openai.com/research/language-unsupervised>.
15. **Vaswani, A., N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. u. Kaiser, and I. Polosukhin**, Attention is all you need. In **I. Guyon, U. V. Luxburg, S. Bengio, H. Wallach, R. Fergus, S. Vishwanathan, and R. Garnett** (eds.), *Advances in Neural Information Processing Systems*, volume 30. Curran Associates, Inc., 2017.
16. **Wu, Y. and P. Yang** (2015). Chebyshev polynomials, moment matching, and optimal estimation of the unseen.